hashlock.

2024

Penetration Test

MyShell

07_06_2024 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

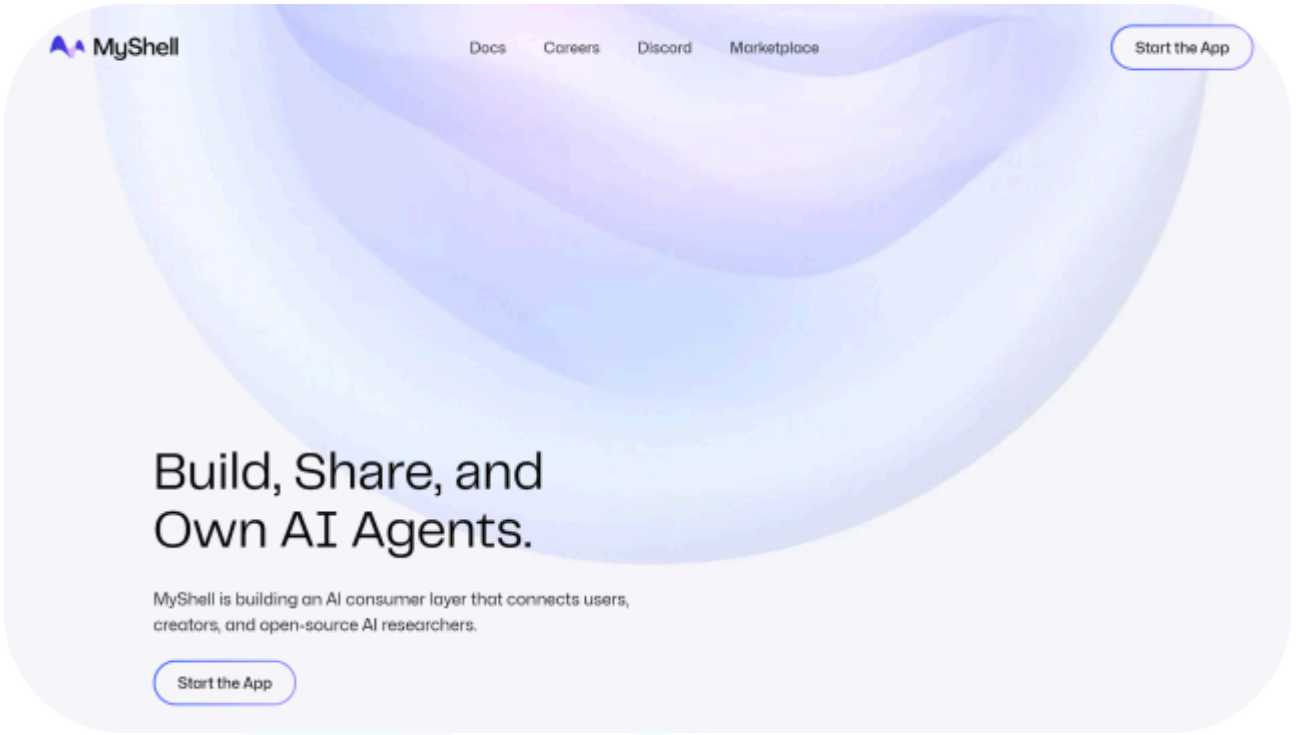
The MyShell team partnered with Hashlock to conduct a penetration testing audit of their website. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed website is secure.

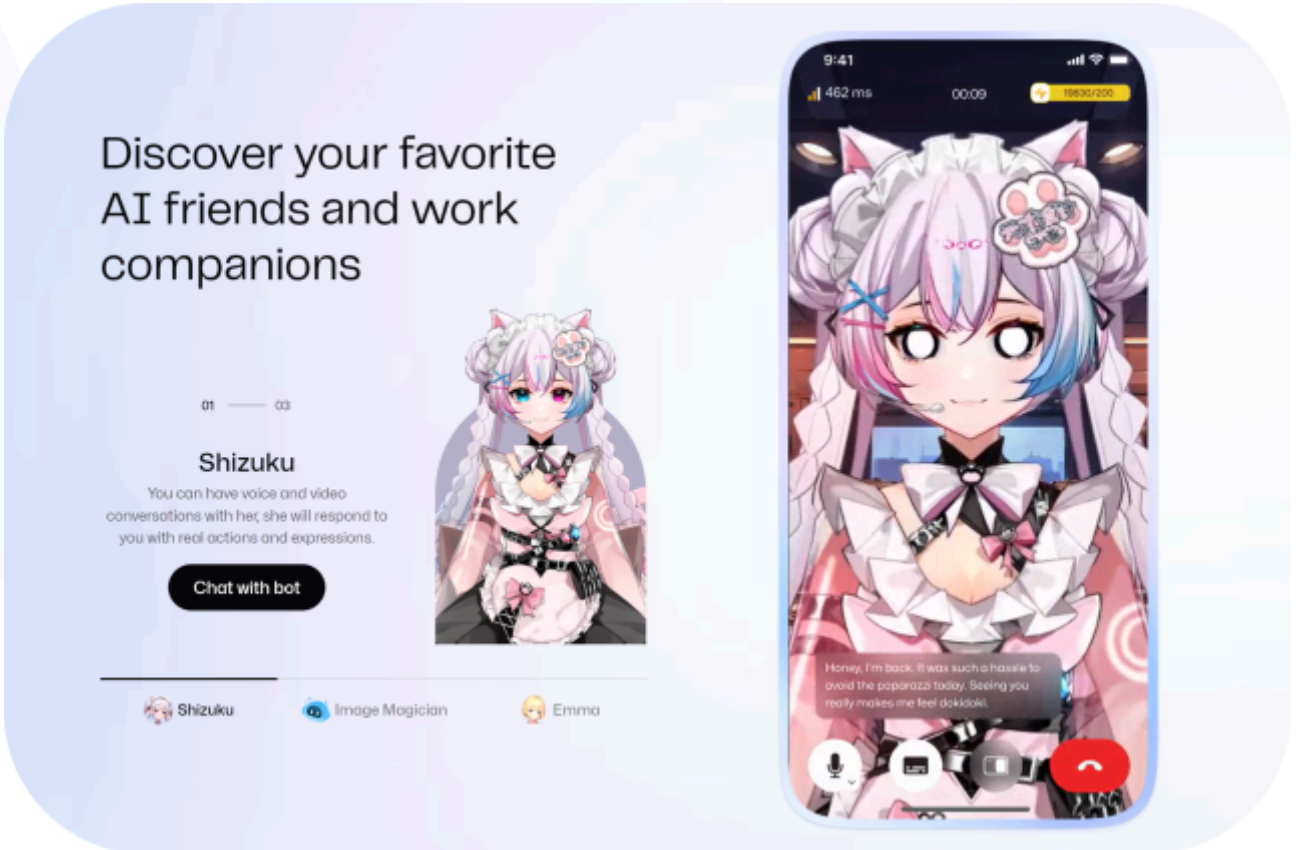
Project Context

MyShell is a platform focused on building an AI consumer layer that connects users, creators, and open-source AI researchers. It allows users to discover AI companions for various interactions, including voice and video conversations. MyShell leverages generative AI models to enable anyone to create AI-native apps, fostering a creator economy where individuals can own and be rewarded for their work. The platform supports AI developers in making their models accessible to creators, contributing to an expanding ecosystem.

Project Name: MyShell
Website: https://myshell.ai/
Logo:

Project Visuals:





Audit scope

We at Hashlock did a penetration test audit for the website of the MyShell team (https://myshelldev.vercel.app), the scope of work included a comprehensive penetration test of their website. We checked their website for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Security Rating

After Hashlock's Audit, we found the website to be **"Secure"**.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High-severity vulnerabilities

1 Medium-severity vulnerabilities

1 Low-severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Code Quality

This Audit scope involves the penetration testing of the MyShell team website, as outlined in the Audit Scope section. All libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the MyShell penetration testing in the form of a website testing environment.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help us understand the overall architecture of the protocol.

Dependencies

Per our observation, the code and libraries used in the infrastructure are based on well-known industry-standard open-source projects.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.

Audit Findings

High

[H-01] Submit.tsx#apiHandler().post for submit endpoint - Malicious sponsored users can make the backend issue unlimited transactions

Description

For sponsored users fees associated with redeeming the airdropped funds are covered by the project itself. The endpoint that handles this case does not check if the user has already claimed their airdrop. Although this check is present in the web3 contract a user can repeatedly send claim requests to the backend and trigger transactions that will fail.

Vulnerability Details

Api handler for `submit` endpoint does not verify if the user already claimed their airdrop

Proof of Concept

This request can be sent repeatedly and the backend will initiate a transaction each time:

```
POST /api/submit HTTP/2

Host: myshelldev.vercel.app

Cookie: < … >

Content-Length: 56

Sec-Ch-Ua: "Chromium";v="125", "Not.A/Brand";v="24"

Accept: application/json, text/plain, */*

Content-Type: application/json

Sec-Ch-Ua-Mobile: ?0

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.6422.112 Safari/537.36
```

```
Sec-Ch-Ua-Platform: "macOS"

Origin: https://myshelldev.vercel.app

Sec-Fetch-Site: same-origin

Sec-Fetch-Mode: cors

Sec-Fetch-Dest: empty

Referer: https://myshelldev.vercel.app/dashboard

Accept-Encoding: gzip, deflate, br

Accept-Language: en-US,en;q=0.9

Priority: u=1, i



{"address":"0xfa9c24780c924f25b805DB3A641311937A2166d6"}
```

Impact

A technical problem or actions from a malicious user can cause the backend to send multiple unnecessary transactions resulting in loss of funds in the form of gas.

Recommendation

Backend should check the on chain data to make sure the airdrop by this particular address was not claimed before initiating a transaction.

Status

Resolved

Medium

[M-02] .env - Insecure Private Key Storage

Description

For sponsored users fees associated with redeeming the airdropped funds are covered by the project itself. In order to do this the backend system uses a private key and an API key to create Biconomy transactions. Currently the these are stored in an env file and are fed to the backend process using environment variables which might lead to funds loss.

Vulnerability Details

submit.tsx file uses the process environment variables to get the private keys:

```
const chainSettings: ChainSettings = isTest
  ? {
      chainId: 11155111,
      paymasterURL:
`https://paymaster.biconomy.io/api/v1/11155111/${process.env.PAYMASTER_API_KEY}`,
      bundlerUrl:
`https://bundler.biconomy.io/api/v2/${11155111}/nJPK7B3ru.dd7f7861-190d-41bd-af80-6877
f74b8f44`,
      rpcUrl: `https://sepolia.infura.io/v3/${process.env.INFURA_API_KEY}`,
      privateKey: process.env.PRIVATE_KEY!,
  } : {
      chainId: 1,
      paymasterURL:
`https://paymaster.biconomy.io/api/v1/1/${process.env.PAYMASTER_API_KEY}`,
      bundlerUrl:
`https://bundler.biconomy.io/api/v2/${1}/nJPK7B3ru.dd7f7861-190d-41bd-af80-6877f74b8f4
4`,
```

```
    rpcUrl: `https://mainnet.infura.io/v3/${process.env.INFURA_API_KEY}`,
    privateKey: process.env.PRIVATE_KEY!,
};
```

Impact

Storing the private keys in an env file and in the process environment variables might expose it to hackers during a possible breach. This can lead to loss of funds.

Recommendation

It's recommended to use secure systems like AWS Key Management System to distribute private keys safely.

Status

Resolved

Low

[L-01] Submit.tsx#apiHandler().post for submit endpoint - Race condition in submit endpoint may lead to wasted gas.

Description

For sponsored users fees associated with redeeming the airdropped funds are covered by the project itself. The endpoint that handles this case currently does check if an airdrop was already claimed, however, it does not have proper locks. Simultaneous claim requests to the backend will trigger multiple transactions.

Vulnerability Details

API handler for `submit` checks currently do not use any locking systems to ensure the same address does not create concurrent transactions while claim status on the blockchain is still unchanged.

```
const isClaimedAlready = await airdrop.usedClaims(leaf);
if (isClaimedAlready) {
```

```
        return res.status(401).send({ result: "You have already claimed this airdrop" });
    }
```

Proof of Concept

This request can be sent using race condition tools to create multiple transactions:

```
POST /api/submit HTTP/2

Host: myshelldev.vercel.app

Cookie: < … >

Content-Length: 56

Sec-Ch-Ua: "Chromium";v="125", "Not.A/Brand";v="24"

Accept: application/json, text/plain, */*

Content-Type: application/json

Sec-Ch-Ua-Mobile: ?0

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.6422.112 Safari/537.36

Sec-Ch-Ua-Platform: "macOS"

Origin: https://myshelldev.vercel.app

Sec-Fetch-Site: same-origin

Sec-Fetch-Mode: cors

Sec-Fetch-Dest: empty

Referer: https://myshelldev.vercel.app/dashboard

Accept-Encoding: gzip, deflate, br

Accept-Language: en-US,en;q=0.9

Priority: u=1, i



{"address":"0xfa9c24780c924f25b805DB3A641311937A2166d6"}
```

Impact

A technical problem or actions from a malicious user can cause the backend to send multiple unnecessary transactions resulting in loss of funds in the form of gas.

Recommendation

The backend should implement locking mechanisms to prevent race conditions.

Status

Resolved

Conclusion

After Hashlocks analysis, the MyShell project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged to achieve security. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the website under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the website in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the website code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the website. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this website.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

The websites are deployed and executed on a web hosting platform. The platform, its programming language, and other software related to the website can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited website.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au


#Hashlock.